TGV Delay Analyst: note to the teachers

Final project, Coding for AI & Data Science (PGE3, 2026–2027) · Bouadi Kheireddine · `https://github.com/BOUADEIKHEIREDDINE/tgv-delay-analyst` · bricks **1, 3, 6, 8, 9**

What it is. You ask a question about French TGV punctuality (in English or French) and get an answer where **every number was computed by code** on SNCF open data. The model is **Qwen3.5-4B Q4_K_M, local in Ollama** (16K context, `Modelfile`); no cloud model, no API key. The harness around it: tools that compute on the data, a system prompt with the real column names, and a guard that blocks any answer containing a number no tool printed.
This note says, for each requirement and each brick, which file, function and screen to look at, and the command that shows it. Longer version with trace excerpts: `EVIDENCE.md`.

Check in 5 minutes

```
git clone https://github.com/BOUADEIKHEIREDDINE/tgv-delay-analyst.git && cd tgv-delay-analyst
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .
python -m pytest                        # expected: 287 passed (~15 s, no model needed)

ollama pull qwen3.5:4b-q4_K_M && ollama create qwen3.5-4b-16k -f Modelfile
python scripts/download_data.py && python scripts/clean_data.py
tgv                                     # the app; /help, /trace, /log inside
```

Question to try: *Quel est le taux de retard à l'arrivée de Paris Lyon vers Marseille St Charles en 2024 ?* → 9.5 %, 95 % interval [8.9 %, 10.3 %]. An answer takes about 30–90 s on my laptop (RTX 3050 Ti 4 GB, thinking on).

The four requirements

Requirement	Where to look	Command / what you should see
R1 · It runs	`README.md` (starts with the title; "Model", "Setup", "Run"), `Modelfile`, pinned `requirements.txt`, `.gitignore` (data, `.env`). Data downloaded by a script, not stored. No key, password or personal data in the repo.	The 5-minute block above. I tested it myself on a fresh clone (2026-11-02): every step worked and the cleaning gave exactly the counts of the committed log.
R2 · It is verified	`tests/`: 13 files, 287 known-answer tests on all my code. The model is replaced by a fake that replays real Qwen replies (`test_agent.py`), the network by a fake (`test_llm.py`), the data by small files in `tests/fixtures/`. Edge cases: empty, missing value, single value.	`python -m pytest` → **287 passed**. Passed on the fresh clone. The tests found and fixed 4 real bugs (listed in `ai_log/journal.md`).
R3 · AI journal	`ai_log/journal.md`: one note per work session (kept as is / corrected by hand / prompt rewritten: which line, why). `ai_log/conversations/full-*.md`: full exports of the 4 sessions (made by `ai_log/export_transcript.py`, personal data removed); short summaries in `2026-*.md`.	Example of a rewritten prompt line, with its measurement: `prompts.py` rules 2, 3, 6, 7 took the test question from 0/3 to 10/10 correct (journal, 2026-11-02).
R4 · Defense	Demo: `tgv`, one question, then `/trace` and `/log`. Code worth asking me about: `agent.run_agent`, `guard.check_answer`, `tools.run_python`, `clean_data.STEPS`.	I can also break the guard on purpose and show the test going red (brick 3 below).

The five bricks

Brick 1: Own tool

Code	`src/tgv_analyst/tools.py` : `run_python` (l. 48, runs Qwen's pandas code in a separate process, 30 s limit, returns what it printed), `describe_dataset` (l. 82), `confidence_interval` (l. 99), `TOOL_SCHEMAS` (l. 114, what the model sees), `call_tool` (l. 162). Called from my own loop, `agent.run_agent` (l. 130).
In the app	Each call appears as a panel "*Tool call [n] · run_python*" (the code Qwen wrote) followed by "*Result [n]*" (what the tool returned). `/trace` shows the last question in full.
Real trace	`benchmark/traces/system.jsonl` (36 real runs). Example `"timestamp": "2026-11-02T18:11:48"` : Qwen calls `run_python` on Paris Est → Strasbourg 2022, the tool prints `6673`, the answer says 6 673 (the true value).
Test	`python -m pytest tests/test_tools.py -v` (26 tests: errors, timeout, forbidden code…)

Brick 3: A rule enforced by code

The rule	**Never show an answer containing a number that no tool printed.** Enforced in code, whatever the model writes: the answer is checked, the model gets one second chance with the list of unverified numbers, then the answer is blocked. Second rule: `run_python` never writes files.
Code	`src/tgv_analyst/guard.py` : `check_answer` (l. 54), `extract_numbers` (l. 23: "12,792", "6 704", "9,5"), `check_code` (l. 83). Called in `agent.run_agent` .
In the app	Badge "*✓ GUARD · n numbers verified*" or "*✗ GUARD · BLOCKED*" under each answer, and "*Sources: [1] call 1 · [2] call 3*": which tool call printed each number.
Tests that try to break it	`tests/test_guard.py` : real Qwen answers ("9,55 %", "9,6 %"), an invented number, an answer with no tool call, `df.to_csv(...)` → all blocked. `tests/test_agent.py::test_invented_number_is_blocked_end_to_end` : fake Qwen writes "9,55 %" twice → "*Answer blocked...*". Known limits written as tests (a correct number used for the wrong thing passes).
Live	In the benchmark the guard rejected 5 answers and the second chance fixed each one, e.g. "6 683" for 6673 (`2026-11-02T19:56:03`), "11,8 %" for 12.0 % (`18:45:40`).
Command	`python -m pytest tests/test_guard.py tests/test_agent.py -v` . To see it go red: remove `"to_csv",` from `FORBIDDEN_CODE` in guard.py and rerun.

Brick 6: Dirty data, written decisions

Data	SNCF open data, *Régularité mensuelle TGV par liaisons (AQST)*, 12,907 rows, licence ODbL. Downloaded by `scripts/download_data.py` .
Cleaning	`scripts/clean_data.py` : 9 small steps listed in `STEPS` (l. 216), each returning the table and one log entry. 12,907 → 12,792 rows. Log: `data/cleaning_log.csv` (e.g. 73 impossible rows removed, 2,183 copied values set to missing).
Rule from a document	`data/RULES.md` , quoting SNCF's page for the dataset: a train is late from 5 min (trip < 1h30), 10 min (1h30–3h), 15 min (> 3h) → `late_threshold` (l. 191), column `seuil_retard` ; and "route figures must not be added up into a national rate". The file also gives the origin of every step.
In the app	`/log` shows the cleaning log.
Command	`python scripts/clean_data.py` · `python -m pytest tests/test_cleaning.py -v` (26 tests, one per step, on a raw fixture with one real anomaly per row).

Brick 8: Numbers with their uncertainty

Interval	Every late rate comes with a 95 % Wilson confidence interval: `src/tgv_analyst/stats.py` `confidence_interval` (l. 7), used by the tool of the same name.
The sentence	Written in code (`ui.DATA_LIMITS`, l. 72), always visible at the bottom of the app (`app.py` l. 225): "*Monthly totals per route: they don't say why trains were late, or whether your train will be. A sum over routes is not SNCF's official national rate.*" More in README, "What the data can't tell" (also: the interval treats trains as independent, so the real uncertainty is wider).
In the app	After the tool result, a bar: "*640 / 6704 · 9.5% ·* ├─●─┤ *· [8.9, 10.3] · 95% CI*"; the limits line under the question box.
Command	`python -m pytest tests/test_stats.py -v` (published values, hand-derived edge cases).

Brick 9: Measured comparison

Setup	My system (Qwen + harness) against my model alone (Qwen, question only): same model, thinking on for both, 12 questions whose answer is computed separately with pandas, 3 runs each (72 runs, 2 h 35).
Result	**Qwen alone 17 % ± 8 %** of answers right (8, 17, 25 % per run) · **Qwen + harness 97 % ± 5 %** (100, 92, 100 %). Read by hand, Qwen alone is even lower: several "right" answers only repeated a number from the question.
Failure explained	`benchmark/results/ANALYSIS.md`: task 4, run 2. Qwen wrote `df.year == 2025 & (...)` without parentheses, the table was always empty, it searched the data instead of its code for 7 turns and hit the turn limit. Nothing false was shown to the user.
Files	`benchmark/tasks.json`, `run_benchmark.py` (`compute_truth` l. 74, `is_correct` l. 86, `summarize` l. 167), `results/results.csv`, `summary.md`, `traces/`.
Command	`python benchmark/run_benchmark.py --summary` (instant, from the saved runs) · `python -m pytest tests/test_benchmark.py -v`

Honest limits

- The guard checks that each number was *printed by a tool*, not that it answers the question.
- The benchmark scoring (same as the kit's `comparer.py`) counts an answer as right if it *contains* the right number; I checked the bare model's answers by hand in `ANALYSIS.md`.
- "Overall" rates are sums over the routes of the dataset, not SNCF's official national rate (SNCF says so; the app says so).
- Copying text out of the app in GNOME Terminal needs `wl-clipboard` (GNOME Terminal ignores the copy code apps use).

AI use: built with Claude Code, directed file by file; every session is exported in `ai_log/`.